

Journal Pre-proof

Prebaking runtime environments to improve the FaaS cold start latency

Daniel Fireman, Paulo Silva, Thiago Emmanuel Pereira, Luis Mafra,
Dalton Valadares



PII: S0167-739X(24)00019-0
DOI: <https://doi.org/10.1016/j.future.2024.01.019>
Reference: FUTURE 7141

To appear in: *Future Generation Computer Systems*

Received date : 14 August 2023
Revised date : 9 December 2023
Accepted date : 14 January 2024

Please cite this article as: D. Fireman, P. Silva, T.E. Pereira et al., Prebaking runtime environments to improve the FaaS cold start latency, *Future Generation Computer Systems* (2024), doi: <https://doi.org/10.1016/j.future.2024.01.019>.

This is a PDF file of an article that has undergone enhancements after acceptance, such as the addition of a cover page and metadata, and formatting for readability, but it is not yet the definitive version of record. This version will undergo additional copyediting, typesetting and review before it is published in its final form, but we are providing this version to give early visibility of the article. Please note that, during the production process, errors may be discovered which could affect the content, and all legal disclaimers that apply to the journal pertain.

© 2024 Elsevier B.V. All rights reserved.

Highlights

Prebaking Runtime Environments to Improve the FaaS Cold Start Latency

Daniel Fireman, Paulo Silva, Thiago Emmanuel Pereira, Luis Mafra, Dalton Valadares

- The Prebaking technique improves the cold start latency in FaaS deployments
- The proposed technique is based on process checkpoint/restore
- The technique improves the cold start latency of three popular runtime environments
- Prebaking provides better cold start latency than SEUSS, starting from slightly complex functions
- Warming up the runtime environment before the checkpoint leads to better results
- The improvement is proportional to the code size and initialization complexity

Prebaking Runtime Environments to Improve the FaaS Cold Start Latency

Daniel Fireman^{a,*}, Paulo Silva^{b,f}, Thiago Emmanuel Pereira^b, Luis Mafra^{b,c}, Dalton Valadares^{d,e}.

^aFederal Institute of Alagoas, Arapiraca, 57317-291, AL, Brazil

^bDepartment of Computing and Systems, Federal University of Campina Grande, Campina Grande, 58429-900, PB, Brazil

^cVTEX Brasil, Praia de Botafogo, 300, Rio de Janeiro, 22250-040, RJ, Brazil

^dMechanical Engineering Department, Federal Institute of Pernambuco, Caruaru, 50740-545, PE, Brazil

^eVirtus RDI Center, Federal University of Campina Grande, Campina Grande, PB, Brazil

^fGoogle, Av. dos Andradas, 3000, Belo Horizonte, 30260-070, MG, Brazil

Abstract

Function-as-service (FaaS) platforms promise a simpler programming model for cloud computing, given that providers take care of the overall resource management while the developers can concentrate only on writing their applications in the scope of a function, not having to care about installing or scaling resources. As FaaS users are billed based on the execution of the functions, platform providers have a natural incentive not to keep idle resources running at the platform's expense. However, this strategy may lead to the *cold start* issue, in which the execution of a function is delayed because there are no ready resources to host the execution. Despite the time to provision the computational resources, starting the runtime environment that manages the function execution can take hundreds of milliseconds to seconds, a prohibitive non-deterministic overhead to latency-critical applications. This work describes a practical technique, *Prebaking*, to reduce the function start-up time based on restoring snapshots of previously executed processes. The foundation of the Prebaking technique is that deciding when to create a snapshot of a function is essential. For example, based on a prototype we developed using the CRIU checkpoint/restore Linux tool, the performance improvement on the start-up of a properly snapshotted function is up to 25 times. Fortunately, generating a good function snapshot is easy: One only needs to checkpoint a warm function, that is, a function that has processed a request. To show the feasibility of using the Prebaking technique, we discussed integrating it into OpenFaaS, an open-source FaaS platform, including how to generate and restore function snapshots. We also evaluated our prototype by running a comprehensive set of experiments that compare the function start-up duration and cold start latency against the standard fork-exec procedure. We analyze the JVM, CPython, and Node.js runtimes. The results indicate that the technique can improve the function replica start-up time even for no-warmed functions: we improved the function replica start-up time from 3.5 times for a "do-nothing" function running in Node.js up to 12 times when considering an Image Resizer function running in CPython. Finally, we reinforce the practicability of the proposed technique by comparing our proposal to another snapshot-based technique called SEUSS. That evaluation shows that Prebaking provides better cold start latency even for slightly complex functions like the Markdown Renderer.

Keywords: serverless computing, faas, runtime environments, cold start, process snapshot, prebaking

PACS: 0000, 1111

2000 MSC: 0000, 1111

1. Introduction

Serverless computing is a new service offering pay-per-use and a low entry barrier to get applications running in a cloud datacenter. A relevant¹ serverless offering is the Function as a Service (FaaS), in which the event handler, the so-called function, is the application deployment and management unit [1]. Thus, the architectural components of a FaaS application must

be abstracted into functions, e.g., HTTP request handlers, usually written in a high-level managed language such as Go, Java, Python, or JavaScript, among others.

The platform is responsible for managing the computing resources used to execute the functions submitted by the users. To this end, the platforms must monitor resource demand and supply. Demand monitoring is based on the requests submitted to a given function. Whenever a request arrives, and there is no resource to execute the request, the resource manager creates a computing resource, which is kept alive for a while after the request finishes. After this keep-alive period finishes and the resource is idle, the resource manager removes the resource.

The increase in serverless computing popularity [2] propitiated the launching of new public platforms in the last few years [3, 4, 5, 6, 7]. One of the reasons for the popularity of serverless computing is that developers can focus on implementing busi-

*Corresponding author

Email addresses: daniel.fireman@ifal.edu.br (Daniel Fireman), pauloifeitosagoogle.com (Paulo Silva), temmanuel@computacao.ufcg.edu.br (Thiago Emmanuel Pereira), luis.mafra@ccc.ufcg.edu.br (Luis Mafra), dalton.valadares@embedded.ufcg.edu.br (Dalton Valadares)

¹<https://www.marketsandmarkets.com/Market-Reports/serverless-architecture-market-64917099.html>

ness logic, leaving resource management and capacity planning details aside.

Another attraction of FaaS is the billing model, which charges users based on the duration it takes for the code to process incoming events or requests. For example, suppose an application is available for a full day but receives only one request and takes 1 second to handle it. In that case, the platform must bill only the 1-second computation performed. Since the users pay only for the duration of function executions, the providers naturally tend to stop unused resources and automatically restart them when needed. On the one hand, that practice leads to the platform saving computing resources. On the other hand, it impacts the application performance via the well-known cold start problem, an unpredictable delay in processing requests observed when the platform has no function instance available to execute them.

The cold start problem happens when a new version of the function runs for the first time and when the FaaS platform policy decides to scale the function up to address a demand growth [8]. In those situations, the user request waits for the platform to launch a new function replica, which includes: i) the time spent provisioning and starting the computing resources to execute the function code; ii) the initialization of the function runtime environment (e.g., the JVM, the Node.js or the Python interpreter); and iii) the execution of application-specific bootstrap, which includes loading and compiling libraries.

Evaluating and improving the efficiency of serverless infrastructure is an active area of research. In particular, research topics related to resource provisioning and its impact on cold starts. A common approach is to avoid delays by being conservative when provisioning functions [9]. For instance, platform providers allow users to configure an idle pool of function replicas, like AWS Lambda Provisioned Concurrency [10]. Other approaches to the cold start performance problem include the usage of specialized sandboxing mechanisms like unikernels [11, 12], lightweight containers [13, 14] and microVMs [15]. Furthermore, some approaches use (micro-)VM snapshots to decrease the function start-up delay [16]. Other strategies control request admission to avoid processing on cold replicas [17].

This work focuses on reducing the runtime environment (RTE) setup and loading delay. We propose a practical technique called *Prebaking*, which replaces the standard fork-exec or the container launch procedures by a mechanism that restores previously created functions processes or container snapshots. We developed a prototype using the Checkpoint/Restore In Userspace tool (CRIU)² to experiment with the performance impact of the technique. We discovered that by correctly choosing the point of the function execution lifetime, the snapshot generated significantly improves the start-up time. Even though the proposed technique is focused on managed languages, it is not restricted to interpreted languages. For instance, Go is a compiled language requiring a runtime environment to manage execution.

In our first experiments (Section 3), we adopted a naive choice for checkpoint: just after the runtime has finished starting. Even for this simple choice, we attained a good speedup.

We evaluated the start-up time of three function implementations executing in three different runtime environments: JVM (Java), CPython (Python), and Node.js. The functions implemented were a NOOP function that does nothing, an Image Resizer, and a renderer of Markdown files. For each function implementation and runtime environment, we compare the start-up time when using the Prebaking mechanism and the usual option based on launching new processes each time a function is started. The results demonstrate that the Prebaking technique is compelling: the start-up time improvements range from 3.5 to 12 times without penalty on the service time.

Following, we observed that creating the checkpoint after processing one request improves the cold start latencies up to 25 times compared to creating the checkpoint just after the function is ready to process requests. That is explained by the fact that processing requests, i.e., warming up the runtime environment, leads to the compilation and optimization of the whole request processing code path. The impact of lazy loading in some runtimes, e.g., Java, significantly impacts start-up delays (Section 4.1).

Following, to foster the discussion among snapshot-based design options, we compared our userspace implementation with SEUSS, an unikernel-based solution [16]. We analyzed the solutions' performance using the NOOP and the function that renders Markdown files. Our results indicate that the Prebaking technique improves the overall latency in the face of cold start from 40% to 270% for the Markdown function.

Finally, we present how to integrate the proposed technique with OpenFaaS, a popular open-source FaaS platform. This proposal shows the feasibility of integrating Prebaking into a production-ready platform, including the checkpoint and restore procedures, without extra complexity. Integrating the Prebaking technique involves changing the function build and deploy processes. At this point, when the user submits a new function to the platform, a checkpoint could be generated to be restored later when a function replica is executed. From the point of view of the platform, the snapshot could be generated during the build after calling a special warm-up request made available by the user of the platform³.

In summary, this paper has the following contributions:

- Dives into the function start-up time to investigate causes of delay, including within the runtime environment;
- Proposes the Prebaking technique, a practical way to improve the FaaS cold starts based on the checkpoint/restore of runtime environment process or container;
- Implements a prototype of the Prebaking technique using CRIU, a checkpoint mechanism available for the Linux kernel;
- Evaluates the Prebaking technique using different functions and runtimes, comparing it with state-of-the-practice and with SEUSS, another checkpoint-based solution;

²<https://criu.org/>

³<https://cloud.google.com/appengine/docs/standard/configuring-warmup-requests>

- Discuss integrating the proposed technique to a popular FaaS platform.

The remainder of this article is the following. In Section 2, we overview the design of FaaS platforms and their relation to the cold start issue. In Section 3, we present the design of the Prebaking technique, its prototype implementation using the CRIU tool, the experimental evaluation design, and the results attained. In Section 4, we show how to improve the gains of the Prebaking technique by checkpointing after warming the runtime environment. Section 5 compares the performance of Prebaking and SEUSS techniques, which harnesses unikernels to execute serverless function snapshots [16]. In Section 6, we show the feasibility of our technique by reporting how to integrate the Prebaking technique into an existing FaaS platform. In Section 7, we overview the literature on FaaS performance improvements, in particular, related to the cold start problem. Finally, in Section 8, we conclude our work and suggest some next steps.

2. Background

2.1. FaaS Platform Architecture

As introduced earlier, the FaaS model has two key aspects: 1) payment is based only on duration and resources used during the execution of functions, and 2) developers are liberated from the operation and management of computing resources. Although all the major FaaS providers share the above basic principles, the design of their platforms is very diverse. Nevertheless, some common, emerging architectural patterns are already identified by the SPEC Research Group on Cloud [18]. We based our background overview on the SPEC-RG reference architecture.

The SPEC-RG reference architecture is organized in three layers: i) the *Resource Orchestration* is responsible for managing computing resources, e.g., containers and VMs, employed to support the executions of functions; ii) the *Function Management* layer, built upon the previous one, is responsible for deploying the function replicas, executing the functions, and autoscaling the function replicas, when needed; and finally, iii) the *Workflow Composition* layer orchestrates multiple functions and manages inter-function state. We focus the remaining discussion on the first two layers since they are more related to the cold start issue considered in this work.

The function lifecycle starts at the Function Management layer when the Function Builder component receives the function source code from the developer and transforms it into a deployable artifact. After the building phase, a deployable artifact is stored in the Function Registry, which can later be downloaded and used to create a Function Replica [18].

The Function Router dispatches new requests or events to the correct function replicas (or queues the requests and events while the replicas are still unavailable to process them). The Function Registry is a repository for the metadata and binaries of the functions available on the platform. The Function Builder transforms the function representations kept by the Function

Registry into a deployable form (which might include compiling, handling dependencies, and other building activities). The Function Deployer drives the deployment mechanisms, deciding how many function replicas should be deployed and which resources should be used to deploy the functions. Finally, the Resource Manager manages available resources and, based on the information gathered by agents running on the infrastructure node, ensures that the computing cluster's state complies with the desired state.

Figure 1 is a sequence diagram illustrating the interaction between some of the reference architecture components to handle the execution of a previously built function with no available replica. That is one of the possible scenarios for the cold start problem. Horizontal arrows depict the direction of the interactions between components starting from an event's arrival. Boxes depict layer components, showing four components from the Function Replica layer and one from the Resource Orchestration layer.

The execution flow shown in Figure 1 starts when the Function Router receives a new event (e.g., HTTP request), which would trigger a Function Replica invocation, but in this case, no replica is available. As a result, the Function Deployer steps in to provide a new Function Replica. To this end, it gathers the desired function configuration from the Function Registry. With the necessary information, the Resource Manager is commanded to deploy the Function Replica on the computing nodes. Once the Function Deployer is informed that a new function replica was created, the Function Router triggers the event that leads to the function execution.

2.2. The Cold Start Problem

Handling the HTTP requests when there are no available replicas takes longer than handling them when there are available replicas. That characterizes the so-called cold start problem. Before being handled by the Function Replica, events hit by cold starts need to wait for two operations to complete: i) the delay in provisioning the computing resources for the new function replica and ii) the delay in the function application start-up.

As provision and virtualization techniques are optimized [14, 19, 20], the runtime environment bootstrap duration and its indirect impact become a more evident problem. Our experiments showed that Java applications typically take more than 150ms to start, and depending on the application initialization requirements, this time can reach 1100ms. Those numbers for code running in Python range from 430 to 480ms, and for Javascript functions, 83 to 150ms.

Furthermore, modern runtime environments postpone some optimizations to when the code is being exercised, and application-specific code might be executed during the function initialization, e.g., connect to databases. Thus, on top of the FaaS platform overhead, a cold Function Replica will potentially deliver worse performance on processing the first request.

3. The Prebaking Technique

In this Section, we describe the design and rationale behind the choice of the checkpoint/restore mechanism used to

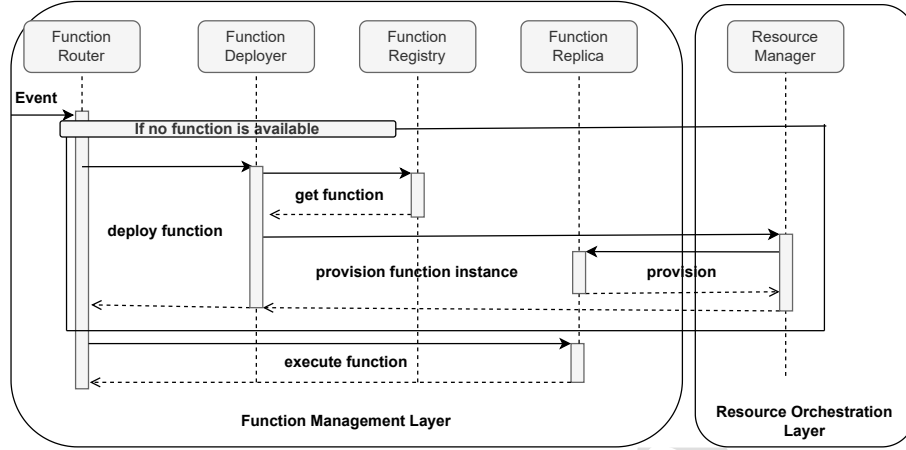


Figure 1: Function execution sequence when a function replica is not available. Adapted from the SPEC reference architecture [18]

prototype the Prebaking technique, which aims to mitigate the serverless cold start problem by accelerating functions' start-up time. Furthermore, the technique aims to: i) be easy to integrate with existing serverless platforms, ii) seamlessly support the most common runtime environments in the FaaS context, and iii) not harm the function performance after the start-up.

This Section also presents the results of a thorough evaluation of the Prebaking technique, comparing its performance with the standard operating system's procedure.

3.1. Design

The Prebaking technique reduces function start-up time by restoring snapshots of previously started functions processes. As shown in Figure 1, in the reference architecture, before a function is ready to serve requests, it executes a complex (and, in many cases, slow) series of steps. These steps include creating a new process to host the runtime, bootstrapping the runtime (e.g., initializing data structures and auxiliary services), and loading the function code. Furthermore, functions might require specific business logic initializations during start-up phase, for instance, configuration load, database, or external service connection pools. We assume that restoring a function snapshot is faster than re-executing all those start-up steps.

The checkpoint/restore technique has been widely used in high-performance computing (HPC) to tolerate faults in long-running applications; when a failure happens, the application can be resumed from periodically generated snapshots instead of restarting from scratch. However useful, checkpoint/restore may impact application performance. On the one hand, the more frequently the snapshots are generated, the faster it is to recover the state just before the failure (because less computation must be re-executed). On the other hand, since the snapshot generation competes for computing resources, frequent snapshots could slow the application down. In the best case, the snapshot generation is pure overhead when there is no fault.

Unlike the usual way checkpoint/restore is used in HPC, the Prebaking technique only creates function snapshots when the user deploys a new function version. As we described earlier, in the reference architecture, the component responsible for transforming the function into deployable artifacts is the Function Builder. For this reason, this component needs to be modified to integrate the Prebaking technique.

A FaaS platform using the Prebaking technique would restore the snapshot whenever a new function instance is created. The same snapshot can be restored to launch different Function Replicas because they all have the same state at the beginning of the execution. More importantly, the Prebaking technique allows the creation of snapshots at any point of the function setup. This characteristic allows for optimizing the snapshot generation process to minimize the restoration delay or other performance aspects. For example, a runtime-agnostic option is to generate the snapshots after the end of the start-up procedure. This alternative eases the snapshot generation. Instead of instrumenting runtime-specific code, it is only necessary to wait for the process to be healthy, e.g., successfully responding to specific TCP requests, to generate the snapshot.

One possible optimization is fine-tuning the runtime environment initialization point to snapshot the function. Choosing the best time requires in-depth knowledge about the runtime environment, such as when the start-up procedure reaches the right balance between progress and the amount of state generated. The rationale behind such optimization is that the larger the snapshot, the longer it takes to be restored.

3.2. Selecting the Checkpoint/Restore Mechanism

The first thing needed to perform the checkpoint is to read the memory and state of the target process. The most straightforward mechanism to perform this is to modify the program to perform its checkpoint. This solution is inadequate for our scenario. As described at the beginning of this Section, we aim

to be easy to integrate with existing serverless platforms, and the solution would demand modifications to the code of any function submitted to the platform. Another possible solution is to use a runtime environment-specific mechanism, like the Coordinated Restore at Checkpoint (CRAc) [21], which is only available for the JVM. Using CRAc in Prebaking is also not ideal, as it would demand specific procedures to support other runtime environments.

A fully transparent checkpoint routine, which executes without acknowledgment of the target program, is doable at the kernel level. At this level, the checkpoint procedure can access the address space of any process. Unfortunately, however doable, the kernel-based solution never achieved mainstream adoption. Another option is to use solutions that relax transparency to stay at the userland, which is the case of `libckpt` [22]. That is still not ideal because it requires the recompilation of the target application to include the code that performs the checkpoint.

The Checkpoint/Restore In Userspace (CRIU) [23] tool can perform a fully transparent process checkpoint at the user level. Instead of modifying the target applications at the compiling time, CRIU injects the checkpoint procedure into the application code while the application is running. After injection, the checkpoint code runs in the same address space of the target process and thus can read its internal state to perform the checkpoint. Once the checkpoint is finished, it removes itself from the code, and the unaware target application resumes execution.

First, CRIU needs to freeze all the target process's threads so that its state does not change while generating the checkpoint dump. With all threads stopped, CRIU discovers what should be checkpointed for each thread. For example, it reads the `/proc/$pid/pagemap` file to find the mapped memory areas. Afterward, CRIU injects the procedure (parasite code) for performing the actual dump into the target process address space using the `ptrace` system call. When the parasite code starts running, it communicates with the CRIU process to know what to dump, reads the content from the process address space, and sends it through a pipe to the CRIU process. Finally, CRIU uses the `ptrace` system call to remove the parasite code and detach itself from the target process, which resumes its execution.

The restore process is more straightforward than the dump one. During the restoration, the CRIU tool process transmutes itself into the checkpointed process. The first action is to read the dump files and restore the process's state. Then, it recreates all namespaces and opened files. Finally, the checkpointed memory is remapped.

Finally, the CRIU package is included in many Linux distributions and is currently integrated with Docker, Podman, and container runtimes. Furthermore, CRIU can run both the checkpoint and the restore mechanisms unprivileged. This is possible due to the recently added `CAP_CHECKPOINT_RESTORE` capability [24]. This capability relaxes some permissions to execute procedures such as selecting a specific pid when cloning a new process and accessing the memory-mapped files.

3.3. Evaluation Design and Setup

The main goal of this evaluation is to assess the performance impact of the Prebaking technique prototyped with CRIU

compared to the fork-exec procedure, which is the default procedure for launching function processes in Linux-based platforms. We focused our evaluation on two questions: *i) Can the Prebaking technique improve the start-up time of a serverless function?* *ii) Does the novel start-up procedure lead to any penalty on the function performance after start-up?*

In our experiments, we varied the start-up method – i.e., **Prebaking** versus the usual start method, based on fork-exec system calls (henceforth, the **Vanilla** method). We measured the runtime environment start-up time and the service time of 600 subsequent invocations. The start-up of a function involves procedures executed by the operating system (e.g., clone, fork, and exec system calls) and procedures executed at the user level. The latter includes the bootstrap of the runtime environment and the application initialization.

Since the application initialization delay is affected by the characteristics of the function, e.g., code size and initialization business logic, we evaluated three different functions: **NOOP**, **Markdown Render** and **Image Resizer**⁴. All functions used HTTP servers to handle the requests, as usually employed in commercial FaaS providers, such as AWS Lambda, Google Cloud Function, Azure Functions, and IBM OpenWhisk [25, 26].

The NOOP function is very straightforward. It does nothing and returns success to every incoming request. The function business logic neither has extra dependencies nor adds extra processing/memory overhead. On the other hand, the Markdown Render converts a markdown to an HTML page. We embed a markdown⁵ inside the body of each incoming request and receive the HTML page as a response. In turn, the Image Resizer is more complex [14, 11]. On start-up, it loads a 1MB, 3440x1440 pixels image⁶, and for each incoming request, the function scales the loaded image down to 10% of its original size.

The last factor was selected because the runtime (RTE) bootstrap delay varies depending on the implementation and to verify whether the solution works seamlessly across RTEs. Thus, we evaluated the start-up of each function using three different RTEs. The functions written in Java, Python, and Javascript had their execution supported by **JVM**, **CPython**, and **Node.js**, respectively.

As we focus on the function start-up, the experiments were composed only of the load generator and the function RTE (i.e., JVM). That means we deliberately excluded some typical components of FaaS platforms, such as container orchestrators [25]. Without loss of generality, focusing on the RTE simplified the experimental setup and removed sources of experimental noise.

The most common concurrent model in public clouds is that each function replica handles one request at a time. If a replica is busy and a new request arrives, the platform starts another replica to do the job. On the other hand, if a replica is inactive for a certain period, the platform reclaims the function replica to save resources [27]. So, to mimic this behavior, the load gen-

⁴<https://github.com/paulofelipefeitos/serverless-handlers>

⁵<https://github.com/PrincetonUniversity/openpilot>

⁶<https://i.imgur.com/Bh1DUOR.jpg>

erator starts the function replica and holds the first request until the replica becomes ready. After that, the load is sent sequentially and at a constant rate.

Figure 2 illustrates the components involved in the Prebaking technique’s evaluation, which include:

- Load generator - responsible for requesting the function restoration and triggering a sequence of requests to be processed. During the request initialization and processing phase, the load generator measures the time needed for the function to accept the first request, as well as the service time of requests;
- CRIU tool - responsible for recovering the function’s process using its snapshot;
- Snapshot - an image of the function’s process previously saved on disk through the Prebaking technique.

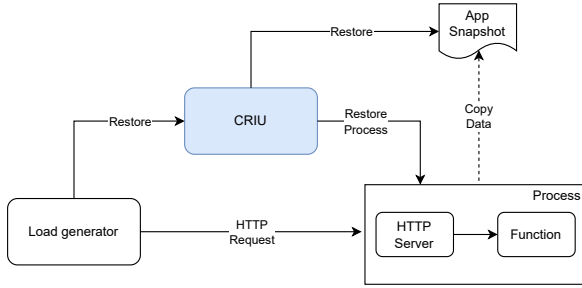


Figure 2: Experiment components and communication flow.

All the experiments were performed in a virtual machine with eight vCPU, 32GB RAM, HDD, and running Ubuntu 18.04. The runtimes were Open JDK 1.8, CPython 3.5.2, and Node.js 17.0.1. Each experiment treatment was repeated 600 times. The load generator and the function runtime were restarted before a run.

3.4. Results

Figure 3 shows a graph comparing start-up times of the Prebaking and Vanilla methods with three different functions and RTEs. Each dot represents one experiment repetition. The median interval for 95% statistical confidence was calculated using the bootstrap method [28], and the error bars and text indicate this interval. It is important to note that there is no overlap between the confidence intervals, indicating that the Prebaking technique results in shorter start-up times in all cases.

First, we found that the Prebaking technique impacted the CPython runtime environment the most. Regardless of the function, it reduced the start-up time by 12 to 13 times. Those results confirm that restoring the snapshot function snapshot from the disk is faster than running all the necessary operations to bring the RTE up and let the application be ready to handle requests.

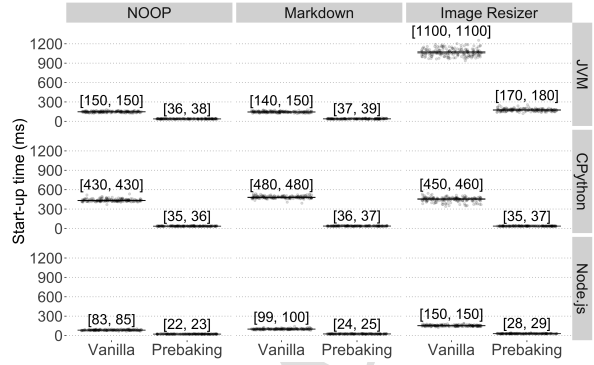


Figure 3: Comparison of the Prebaking and Vanilla start-up times.

Surprisingly, as the function complexity increased, the RTE bootstrap duration of CPython remained unaffected. The Image Resizer function uses a file to emulate business logic during RTE initialization, but operations over the file are lazy. In more detail, the Image Resizer implementation uses the Python Image Library’s open method⁷; the file is opened during RTE bootstrap, but the image data is not loaded on memory until the resize, which happens at request handling.

The JVM and Node.js results show similar trends: the higher the function complexity, the higher the start-up time using the Vanilla technique, and the higher the performance benefit of using the Prebaking technique. The improvements range from ≈ 4 to 6 times in Java and 3.7 to 5.3 in Javascript. One point worth noticing is the increase in Image Resizer’s snapshot recovery duration for the JVM. We believe this is because the Java library used to resize the image⁸ stores the decoded image in memory.

Finally, we look for statistical confidence to answer the first research question. As some samples failed the Shapiro-Wilk normality test [29], we used the non-parametric Wilcoxon-Mann-Whitney Test to verify whether using the Prebaking technique leads to a significant difference in the start-up time median. The tests confirm, with 95% of statistical confidence, the benefit of the proposed technique in all considered cases. Thus, the Prebaking technique leads to shorter start-up times and thus decreases the function latency variability.

In the state of the practice, those improvements imply more latency predictability, as the user-perceived latency of the first request suffers less from the cold start. As we described earlier, the function warm-up is executed during the build process. Thus, the warm-up is not the function execution hot-path. A remaining concern is how functions behave after being restored, leading us to the second question (*Does the novel start-up procedure lead to any penalty on the function performance after start-up?*). To answer that question, we assess whether selecting the Prebaking technique to launch the function runtime en-

⁷<https://pillow.readthedocs.io/en/stable/reference/Image.html#PIL.Image.open>

⁸<https://docs.oracle.com/javase/8/docs/api/java/awt/image/BufferedImage.html>

vironment affects the service time.

Figure 4 presents the empirical cumulative distribution function (ECDF) of the service time for 600 requests applied to all functions implementations and platforms after being initialized by the Prebaking and Vanilla techniques. The ECDFs for the Vanilla and Prebaking techniques coincide for each configuration of function and runtime (e.g. NOOP on JVM). Thus, it indicates that the Prebaking technique does not lead to any performance penalty after the functions are restored, regardless of the runtime environment.

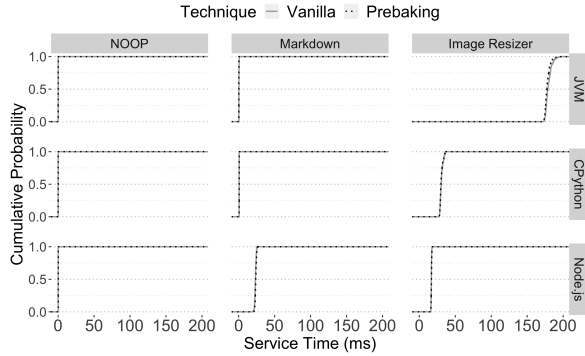


Figure 4: Service time ECDFs of Prebaking and Vanilla. Empirical service times for 600 requests sent to NOOP, Markdown Render, and Image Resizer functions implementations executed by the JVM, CPython, and Node.js runtime environments.

4. Dealing with the Cold Runtime Performance Impact

After we evaluated prebaking as a feasible strategy, which improved the RTE start-up time in all considered cases, we conducted another evaluation to understand what happens within the RTE when the cold start issue occurs. From the function caller’s perspective, that means waiting for the RTE to bootstrap, the application to be loaded and executed, and the request service time (ST). We call that **cold start latency**. As a lot happens during the RTE start-up, we split it into four components (or phases): i) execution of the `clone` system call (CLONE), ii) execution of the `exec` system call (EXEC), iii) the period between the end of the `exec` call and the start of the `main()` procedure (runtime start-up - RTS) and iv) from the end of the RTS phase to when the function is ready to serve the first request (application initialization - APPINIT).

To measure the duration of the first two start-up phases, we instrumented the execution of the application using the `bpfttrace` tool⁹. We introduced logs using the standard system call probes available in `bpfttrace` (in which one can probe both the entry and exit points for any system call). We added logs to measure the last two phases before the runtime started executing the first line of code and right before the HTTP server handled the first request. We repeated each experiment 600 times.

⁹<https://github.com/iovisor/bpfttrace>

Figure 5 depicts the cold start latency components of the NOOP and Image Resizer functions running on all three considered RTEs using the Vanilla approach. It presents the proportion of the duration of each phase as a part of the overall latency. For instance, Figure 5 allows us to spot that, regardless of the case, `clone` and `exec` system calls contribute to a negligible fraction of the start-up time.

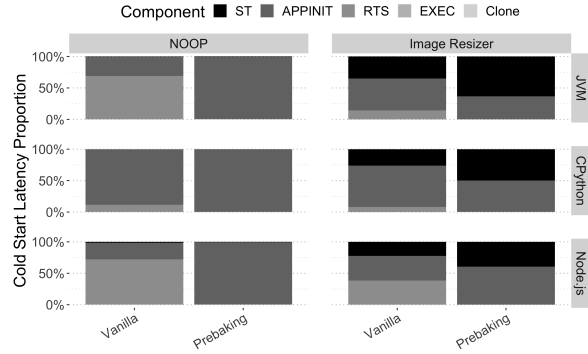


Figure 5: The breakdown of cold start latency components for the Vanilla approach in three runtime environments. It is essential to note that the “CLONE” and “EXEC” constitute only a minor fraction of the total start-up time.

It is important to notice the different start-up profiles among the different Vanilla cases. On the one hand, the JVM and Node.js bootstraps spend most of their time in the RTS phase, whereas the CPython is in the APPINIT. That is due to the runtime-specific characteristics, for instance, the JIT operations performed by Node.js and JVM. On the other hand, the Image Resizer function business logic changes that proportion by loading the 1MB image file at bootstrap, increasing the duration of the APPINIT phase.

The Prebaking technique reduces the RTS to 0ms, regardless of the function. Therefore, the APPINIT and ST phases dictate the overall start-up time. It was anticipated, given that the technique involves creating an image of the running function runtime that can be loaded on demand. Furthermore, its start-up is also impacted by RTE and application specifics. The loaded image increases the snapshot, leading to a slower recovery. Nonetheless, our previous results demonstrate that it is always faster than launching the runtime using the Vanilla technique.

Finally, it is well known that the service time is also impacted by runtime specifics, e.g., dynamic class loading and just-in-time (JIT) [30]. At the start, the function’s first request may take longer due to the need for dynamic compilation. However, the optimizer will eventually reach a steady state or “**warm up**”.

4.1. Warming up the Runtime Environment before Prebaking

Those valuable lessons learned so far led us to the idea that the Prebaking technique could be used to eliminate runtime-specific overheads and improve the APPINIT and ST cold latency components. Instead of baking a freshly started runtime

environment image, one could warm it up before baking. We called this technique **Prebaking Warm**. A warm-up procedure example is sending relevant requests to the serverless function, which triggers the dynamic code compilation and other optimizations.

We evaluated the idea through the following research questions: *can the Prebaking Warm technique reduce cold start latency?* To answer that question, we ran a series of experiments comparing the performance under cold starts of the Vanilla, Prebaking, and Prebaking Warm techniques. To assess how sensitive the runtime mechanics are to the growth of function code and its impact on the snapshot generated, we implemented JVM, CPython, and Node.js versions of a function that loads a predefined number of classes/modules when invoked and selected three flavors: i) small - $\approx 2.8\text{MB}$, ii) medium - $\approx 9.2\text{MB}$, and iii) big - $\approx 41\text{MB}$. We measured the cold start latency and repeated each treatment 600 times.

Before answering the research question, it is fundamental to understand how the performance of different runtimes is impacted by the increase in code complexity using the current state of the practice, i.e., the Vanilla launch technique. Figure 6 summarizes these results and demonstrates that the cold start latency increases with the function code size for all three runtimes.

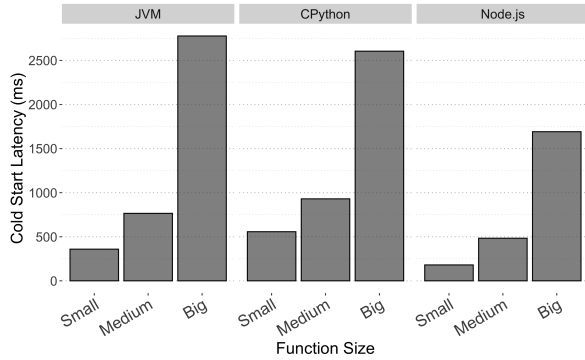


Figure 6: Impact of the function size on the function cold start latency.

To answer our research question, we analyzed Figure 7, which shows the difference in cold start latency between the Vanilla technique and both versions of the Prebaking technique. Our results indicate that warming the function before baking improves its cold start latency. We observed a minor improvement in Small functions running in Node.js, where the Prebaking technique improved the latency by 157.7%, while the Prebaking Warm technique improved it by 538% (3.4 times). For the same function running in JVM, the latency gains increased by 4.5 times. For Medium and Big functions, the CPython runtime showed the most significant gains, while the Node.js runtime presented smaller but still significant gains across all functions. Moreover, we found that the performance gain increases with the complexity of the code, as the snapshot loading is less impacted by function size compared to the Vanilla source-code loading and compilation.

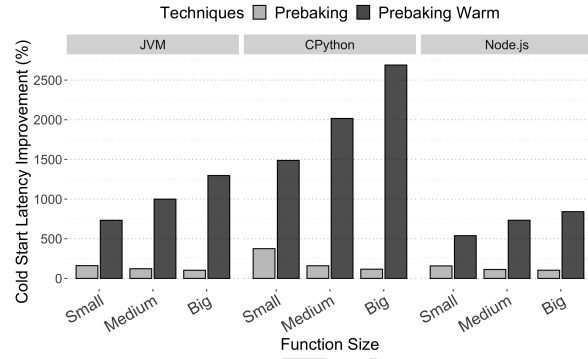


Figure 7: The cold start latency improvement of both Prebaking techniques compared to Vanilla. The Prebaking Warm improvement shows the impact of warming the functions before generating the snapshots.

Finally, table 1 helps us analyze this impact by showing the cold start latency intervals for the three start-up techniques, function sizes, and runtimes. Prebaking Warm is better than Prebaking in all cases, and none of the CI intervals intersect, which answers the research question with high statistical confidence.

5. Comparing Snapshot-based Approaches

In this Section, we compare Prebaking to another technique called Serverless Execution via Unikernel SnapShot (SEUSS), which also aims to reduce serverless cold start latency. We will explain the SEUSS technique, how it differs from Prebaking, and the evaluation methodology used. Finally, we will present and discuss the results.

5.1. Serverless Execution via Unikernel SnapShot (SEUSS)

SEUSS (Serverless Execution via Unikernel SnapShot) is another snapshot-based method that aims to accelerate the implementation and decrease the memory footprint of serverless applications [16]. Adopting unikernels to isolate the application execution is one of the reasons for the significant reduction in cold start latency. Unikernels are specialized kernels that offer operating system components as libraries. They combine the application and kernel in the same memory address space, creating an executable that can initialize directly on virtualized hardware. Even though it makes building harder, this strategy is advantageous for cloud applications as it eliminates several unnecessary software layers designed for general-purpose operating systems. By reducing the amount of code to deploy, the start-up time is shorter, and the application's security and performance are enhanced.

The SEUSS method takes advantage of the quick unikernel boot and stores the unikernel's execution state in cache memory. The execution state considers various steps of invoking a function. These snapshots are called UC (Unikernel Context). The objective of the UC strategy is to avoid performing the same start-up steps while creating new instances.

Table 1: Cold start latency intervals (in milliseconds) for functions with small, medium, and big code bases. Confidence intervals were calculated to provide 95% of statistical confidence.

	JVM			CPython			Node.js		
	Vanilla	PB	PB Warm	Vanilla	PB	PB Warm	Vanilla	PB	PB Warm
Small	(360;360)	(220;220)	(49;50)	(550;560)	(150;150)	(37;38)	(180;180)	(110;120)	(33;34)
Medium	(760;770)	(620;630)	(76;77)	(930;930)	(580;580)	(46;47)	(480;490)	(430;430)	(65;66)
Big	(2800;2800)	(2700;2700)	(210;220)	(2600;2600)	(2200;2200)	(96;98)	(1700;1700)	(1600;1600)	(200;200)

The SEUSS method proposes that an initialization process must be performed when a new compute node joins the platform’s computational resources. This process avoids the start-up delay of the runtime and isolated execution environment. For example, for every runtime supported by the platform, a unikernel binary must be run, containing the runtime code and an invocation script. The script is responsible for creating an HTTP server and importing the function source code when necessary. After the boot process, SEUSS takes a snapshot, creating a UC. Since the UC has not loaded any function yet at the snapshot moment, it can be used to create function replicas that share the same runtime.

That behavior leads to three possible cold invocation types: cold, warm, and hot. When an invocation falls under the cold path, it requires all initialization steps, including providing the isolated execution environment, as described in Section 1. That path should not happen in practice as the runtime-UC is created when a new compute node is added to the platform. In the warm path, the method assumes that the function’s execution will be delayed because the function’s source code has yet to be loaded and compiled. Lastly, in the hot path, the function has already undergone all the start-up steps, and only a UC snapshot restoration is required to process the request.

Even though the strategy solves the cold path, dealing with the warm path is also essential. The process of importing and compiling the source code of a function and its dependencies can take as long as starting a container or VM. To address this problem, SEUSS defines that whenever a function is invoked, and only the UC with the generic runtime is available, the procedure must restore the UC and perform the entire function initialization process. This way, when the function is ready to process requests, SEUSS takes its snapshot, which will cache a UC that avoids all the initialization steps. That approach is similar to what happens in the Prebaking technique.

By combining all the strategies to deal with the cold and warm paths, SEUSS can check which UC has the most advanced initialization state available in the cache whenever necessary to create a new function replica. Then, it can restore the UC state to deploy the new function instance, avoiding start-up steps.

5.2. Prebaking and SEUSS Differences

The main difference between the Prebaking strategy and the SEUSS is that Prebaking only takes one application snapshot. The technique defines that the snapshot should be generated when the function completes the initialization process. Furthermore, the snapshot must be created when a new version of the

function code is deployed on the platform and not at its invocation, as the SEUSS method proposes.

SEUSS chose this strategy to reduce the memory quantity used by serverless applications. Performing snapshots at different moments of a function’s life cycle facilitates sharing memory pages between UCs through the copy-on-write mechanism.

On the other hand, the consequence of taking the snapshot during the function invocation is that the functions will still have to pay the price of loading and compiling the source code. Despite saving a UC after this process, this cold-start event happens whenever there is no replica available or no snapshot with the function ready to run. Besides, functions with many dependencies can increase start-up time to a few seconds.

Another difference between the two strategies is that the SEUSS method uses unikernels to isolate the execution of functions. However, unikernels need a specialized operating system implementation, meaning the platform compute nodes are not hosted by a Linux operating system. Despite all the unikernel advantages, the consequence of its use can make the SEUSS integration more complex with serverless platforms, making its adoption even more difficult.

5.3. Evaluation Design and Setup

This Section describes our methodology to compare the performance impact of Prebaking against the SEUSS alternative. Mostly, the methodology was similar to the methods used in the experiments described in previous sections. We varied the start-up method – Prebaking versus SEUSS. We measured the cold start latency, which accounts for the duration between the user invoking the request and obtaining the response. We evaluated two different functions, described in Section 3.3: **NOOP** and **Markdown Renderer**. In this experiment, we considered Node.js implementations for both the two functions^{10,11}. We repeated the execution of each function 100 times, with sequential requests sent at a constant rate in each run.

To foster reproducibility, we adopted the original SEUSS tooling to measure its performance. The tooling includes the “SEUSS Hosted” and “SEUSS Native” components. The SEUSS Hosted component implements a load generator and collects the function’s initialization delays and the service times for each request. During the experiment, the SEUSS Hosted load generator sends the code of functions to the SEUSS Native component

¹⁰<https://github.com/paulofelipefeitosa/serverless-handlers/tree/master/functions/nodejs/>

¹¹<https://github.com/paulofelipefeitosa/serverless-handlers/tree/master/seuss/functions/>

to request the execution of the functions. In its turn, SEUSS Native is responsible for checking if any UC is available to execute the function in the cache or if it is necessary to create a new UC from the NodeJS unikernel. Figure 8 shows the communication flow between SEUSS components.

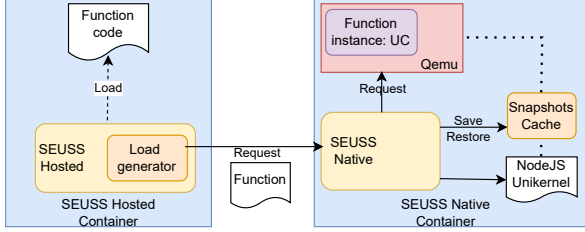


Figure 8: Components and communication flow

5.4. Results

Figure 9 compares the cold start latency in milliseconds for the NOOP and Markdown functions using the Prebaking Warm and SEUSS techniques. The error bars and the text present the median range with 95% statistical confidence using bootstrap [28].

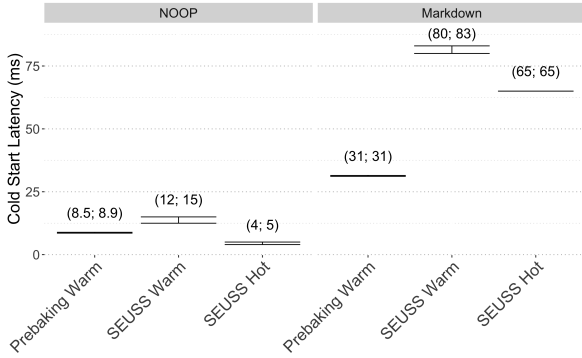


Figure 9: Comparison of the cold start latency from executions using the Prebaking Warm and SEUSS techniques.

Based on the confidence interval of the medians, it is clear that Prebaking Warm outperforms SEUSS when used in warm conditions. The median performance difference for the NOOP function ranges from four to six milliseconds, representing a significant improvement of 33% to 42% with the Prebaking technique. Regarding the Markdown function, there was a significant improvement of over 60% as the cold start latency difference between 49 and 53 milliseconds.

After taking a snapshot of the loaded function and evaluating the NOOP function, the results differ for SEUSS. In cases where only UC snapshot restoration is necessary, SEUSS outperforms Prebaking Warm by around 50%. This is particularly evident when setting up a basic runtime with minimal code. However, when dealing with a more complex function such as the Markdown Renderer, Prebaking Warm proves to be twice as efficient as SEUSS Hot.

Even though we did not go deeper to understand the causes of this performance difference, there are some important considerations to keep in mind:

1. the way SEUSS imports the function's code does not allow the reuse of variables, causing the import of dependencies necessary for initialization to be re-executed at each function invocation;
2. the fact that QEMU, a virtualized hardware environment, executes SEUSS, and the solution was packaged in a container may have contributed to the degradation of the performance of the technique [31, 32, 33];
3. more computational resources might be needed to properly evaluate SEUSS, as used in [16]

6. Integrating Prebaking to a FaaS Platform

To be as generalist as possible, we did not tied-up our Prebaking technique to specific serverless platforms, nor to any virtualization technology used by these platforms, such as Docker[34] and Firecracker[19]. To assess the feasibility of using the Prebaking technique, we integrate it with OpenFaaS¹², one of the most popular open-source serverless platforms. It provides excellent documentation about the platform architecture, making understanding how we could integrate our technique easier.

In the following sections, we overview the OpenFaaS design and explain how it is integrated with the prebaking technique. In particular, we discuss how the OpenFaaS function build operation could be modified to generate snapshots of the functions submitted to the platform. The snapshots are embedded into the Docker images, the virtualization technology used by OpenFaaS, which are later registered to be used by function replicas.

6.1. OpenFaaS

OpenFaaS is a container-based serverless platform. It means there is a container for every function, and the container should encapsulate all dependencies (e.g., source code and runtime). That said, Figure 10 presents an overview of the OpenFaaS architecture and how its components communicate.

As shown in Figure 10, developers operate OpenFaaS through the FaaS-CLI or FaaS-UI, which defines an API for the operations with functions. As an example, we list some operations that are essential to the integration with the Prebaking technique:

1. **new**: creates a new function project by copying a language template from the Templates Repository. After that, the developer can edit the project to implement the function's business logic;
2. **build**: transforms the function source code into a deployable artifact which is a Docker container image;
3. **push**: stores the function deployable artifacts into the Function Registry, which is a Container Image Repository;

¹²<https://www.openfaas.com/>

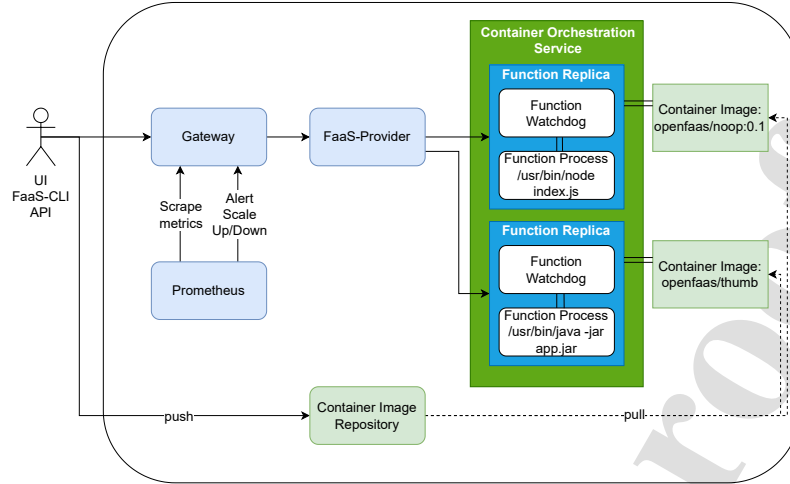


Figure 10: OpenFaaS main components.

4. **deploy**: deploys the function instance into an OpenFaaS Gateway, enabling its usage through the platform.

Every request through the platform hits the Gateway API, the OpenFaaS platform entry point. It provides APIs to deploy, invoke, scale, and gather information and metrics about the instances of the function. Furthermore, the platform auto-scaling functionality is shared between the Gateway API and the Prometheus tool, which continuously monitors metrics and fires alerts. All alerts fired by Prometheus are processed by Gateway API, which decides when to scale down/up the number of active function instances.

Instead of directly executing operations, such as incrementing the number of replicas of a particular function, the API Gateway delegates it to the FaaS-Provider. This indirection abstracts details of different container orchestration mechanisms and tools. Currently, the FaaS-Provider has official implementations for Kubernetes and Docker Swarm¹³ integration.

Finally, the function Watchdog is the component responsible for managing and monitoring the function replica lifecycle. Furthermore, it is a communication interface between the platform API and the replica process.

6.2. Prebaking OpenFaaS Functions

Figure 11 shows the OpenFaaS template concept. A template hides setup complexity from users who have everyday use cases. There are templates for languages like Go, Python, Java, PHP, and C#.

To spin off a prebaked function, we must create a template that adds all CRIU dependencies and executes CRIU commands. As CRIU uses different commands to start processes in different runtimes, we created a new CRIU-version template for each language that we wanted to support¹⁴. With the prebaking template, the developers can create function projects that

adopt the prebaking technique by using the new operation from FaaS-CLI.

Prebaking templates work differently from usual OpenFaaS templates. On the build phase, when transforming the source code and dependencies into a deployable function, Prebaking templates start the function runtime and run an optional post-processing script (e.g., warm-up requests), and checkpoint the function process into the container image. Moreover, after creating the Docker container image, the OpenFaaS platform can finish the function push and deploy processes as usual. When it is time to start the function replica, the container executes the CRIU command to restore the dump previously saved inside the container image.

After creating a new function using the CRIU templates, developers can deploy function instances using the same commands provided by FaaS-CLI.

7. Related Work

Fighting delays in the function start-up process remains relevant for the serverless community. Several alternatives have been proposed to mitigate the impact of long start-up times. In addition to checkpoint/restore methods, as presented in this work, cache-based methods and sandbox optimization were considered [35]. Also, many solutions combine these alternatives to mitigate start-up delays. This section reviews some of these solutions and relates them to our work. We also overview our previous work on the Prebaking technique.

SOCK [14, 36] is a representative example of cache-based optimizations. In SOCK, application loaders are cached with pre-imported, common packages. During start-up, the loaders are cloned to avoid long start-up delays. A limitation of such an approach is that it is language-specific.

In many serverless platforms, container sandboxes isolate and protect function replicas. Despite being light, popular container solutions, such as Docker [34] and LXC [37], might present long start-up delays. Faster and lighter sandbox alternatives

¹³Docker Swarm is deprecated and no longer maintained by OpenFaaS.

¹⁴<https://github.com/paulofelipefeitosa/templates>

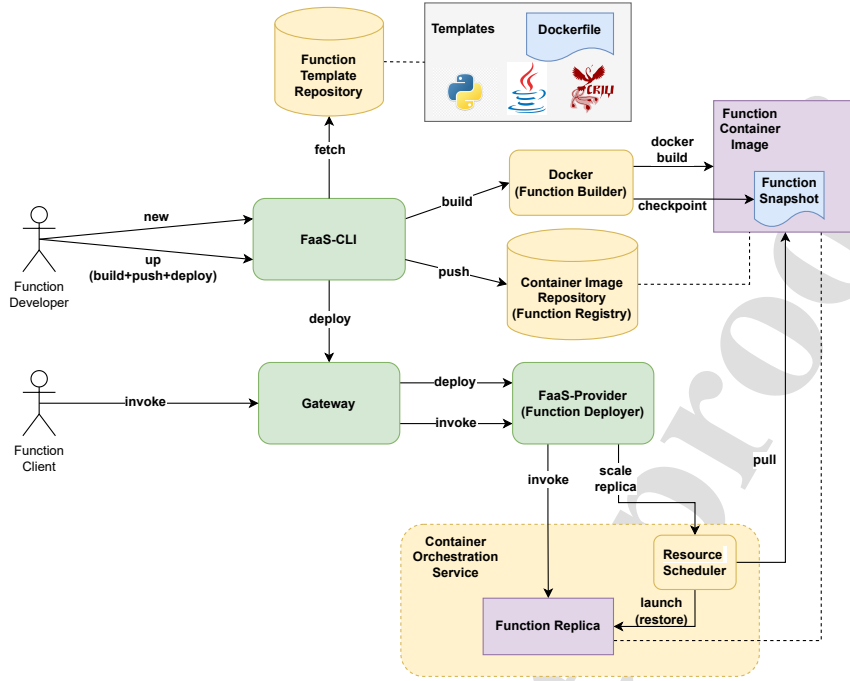


Figure 11: Diagram of the Prebaked Functions deployment and execution flow in the OpenFaaS platform.

were considered both in academia and industry. AWS Firecracker micro-VM is a remarkable case [19].

Boucher *et al.* [38] proposes a deviation from the standard container-based sandboxing. Instead, they adopt isolation based on the Rust runtime. The authors implemented a multi-tenant worker process in Rust, which directly executes functions by dynamically loading the function code and running it as a thread. Even though they achieved a start-up time in the order of microseconds, the solution requires users to write functions in Rust, which brings usage and code transpilation challenges. Our work improves the start-up time of applications without changing the user code or imposing new user requirements.

Another design alternative to containers is Unikernels, as SEUSS adopted (a great example of a combination of caching, sandboxing, and checkpoint/restore techniques) [16]. Unikernels provide fast application execution because the application is statically linked with the kernel. In addition, during the build process of this bundle of application and kernel, any service and library that is not necessary to execute the application could be removed. In addition to the discussion raised in Section 5, it is essential to note that adopting an ad-hoc kernel instead of an out-of-box Linux kernel comes with a price: it leads to more complex integration with existing serverless platforms. In particular, when small organizations deploy the platform on-premise, the operation cost could be prohibitive. Furthermore, how Unikernels performs when dealing with more complex functions needs to be clarified, as the SEUSS work describes only the evaluation of the NOOP function.

The technique of checkpoint/restore of snapshots gained the

industry mainstream only more recently. AWS Snapstart [39] allows to publish a function based on a snapshot of a Firecracker micro-VM containing an already initialized execution environment. When new replicas of the function need to be created, they are resumed from the created snapshots.

Before gaining mainstream, the checkpoint approach had already been used in other contexts before ours [40] and SEUSS [16] serverless proposals. For example, Kukreti and Mueller [41] propose the process cloning technique to avoid speculative tasks recomputing work already done by the original task, improving the probability of a speculative task catching up with the straggler task. Kawachiya *et al.* [42] propose to clone and restore the JVM internal structures, avoiding several start-up steps to reduce Java applications start-ups. Oh and Moon [43] proposes to decrease Web application’s start-up time by snapshotting Javascript objects and restoring them when the application is loaded. We chose the process cloning technique because of its generality. As the clone operation is applied at the process level, it does not need to know runtime or application internal structures. However, the knowledge of runtime internals could be used to make the start-up even better.

Checkpoint/restore was also considered in combination with other serverless techniques. This was the case of Shillaker and Pietzuch [44] FAASM, lightweight isolation for HPC serverless. FAASM isolates the CPU and network, being the runtime for FaaSlets, which use software-fault isolation (SFI) to isolate the executed functions’ memory. FAASM also uses snapshots to reduce initialization times when restoring FaaSlets. The authors performed experiments to compare FAASM with a stan-

dard container-based platform. Results demonstrated that FAASM uses ten times less memory while being two times faster during a machine learning model training and reduces tail latency by 90% while doubling the throughput during the model inference.

Ustiugov et al. [45] presented vHive, a snapshot-based open-source framework for serverless experimentation. The vHive uses Containerd orchestration and Firecracker hypervisor. The authors analyzed the function cold-start delays when starting from snapshots and verified that the frequent page faults when getting the function's state from disk result in high latencies. They also noticed that different invocations of the same function generally access a stable working set of pages. Knowing this, they proposed the REAP (Record And Prefetch) mechanism to mitigate the high latencies problem. REAP records the stable working set of pages during the function start-up and prefetches it from disk to memory when creating new function replicas. Experimental results demonstrated that REAP improves cold-start delays by 3.7 times.

In addition to architectural proposals (caching, snapshotting, and sandboxing), improvements in policies (and configuration optimizations) have been considered. For example, Kim and Roh [46] proposed a container scheduler named FPCSch (which assumes function snapshots). Fuerst and Sharma [47] employed keep-alive policies to keep functions alive and warm, alleviating the cold-start overhead. They implemented caching-based keep-alive policies in a platform based on OpenWhisk named FaasCache. Experiments with FaasCache show that the proposal can decrease the system load time, increasing the number of served requests. Agarwal et al. [48] proposed a reinforcement learning (Q-Learning) method that analyzes CPU consumption to verify invocation patterns and prepare instances in advance, reducing the cold start times.

7.1. Previous Contributions

In a previous work [40], we introduced the Prebaking technique. In the original presentation, we proposed the technique and evaluated its performance in a single JVM runtime environment. Additionally, we demonstrated how to integrate this technique into an open-source FaaS platform. However, two significant issues still need to be addressed. Firstly, how the prebaking technique would perform with snapshot functions in other runtimes needed to be clarified, as each runtime has a unique initialization procedure. It was still determined how prebaking would behave outside Java. This paper addresses this gap by presenting research on the topic. Secondly, the previous work did not provide an experimental comparison of the prebaking technique with other snapshot techniques. Although we had compared the prebaking design with other solutions, we did not conduct an experimental comparison. This paper addresses this issue by comparing performance with a Unikernel-based solution.

8. Conclusions and Future Work

Serverless platform users must confront the notorious problem of high non-deterministic response times when the request

handling requires waiting for the platform to scale up. This problem, commonly known as cold start, has various causes, such as platform orchestration overhead and virtualized environment (container or VM) start-up [20, 9]. However, our findings, corroborating with previous studies [14, 49, 9], reveal that the runtime environments' specifics also play a major role in cold starts. Our experiment results demonstrate that the start-up times of realistic FaaS functions executing in Node.js, CPython, and JVM range from 83ms to 1100ms, depending on the runtime environment and the code.

This work focuses on decreasing the latency related in the face of cold starts (i.e., cold start latency). More precisely, it reduces the impact of specific runtime environment routines executed at start-up and during the process execution. The proposed solution uses a process cloning technique based on Checkpoint/Restore In Userspace (CRIU) tool. We called this technique Prebaking, and it consists of persisting the process state of a warmed ready-to-serve function instance to recover this state in a cloned process later when the platform needs to scale up. The tool is agnostic to the runtime environment and is currently packaged for many Linux distributions, which eases adoption by FaaS platforms.

Our research has demonstrated that implementing Prebaking to launch serverless functions can significantly reduce the time required for the runtime environment to start-up. Our results indicate that Prebaking can improve NOOP function performance by up to four times and, for more representative functions, by up to six times. We observed similar outcomes for both the CPython and Node.js runtimes, with Prebaking leading to 3.5 to 5.3 times improvements in Node.js functions and 3.7 to 12 times in CPython functions. We compared our proposed method to another snapshot-based technique, SEUSS, and found that Prebaking provides better cold start latency for slightly complex functions, such as the Markdown Renderer.

Furthermore, the proposed solution allows users or platform maintainers to interact with the process before snapshotting its state. We used this feature to store warm versions of the runtime environments. That change led to cold start latency improvements ranging from 1.5 to 25 times, depending on the runtime and the code complexity. That means the prebaking technique removed the overhead of the runtime environment procedures and effectively removed the overhead caused by dynamic loading dependencies and compiling the code. We showed that these gains are proportional to the code size of the function.

Finally, we assessed the practicality of the Prebaking method by providing a comprehensive explanation of how to incorporate it into Open FaaS, an open-source FaaS platform.

Moving forward, our team aims to expand the implementation of the Prebaking technique across various FaaS platforms to determine its compatibility with different designs. We also intend to evaluate the checkpoint/restore service, focusing on its ability to handle larger function code sizes and concurrent snapshots efficiently. Lastly, we plan to experiment with in-memory optimization on CRIU to enhance snapshot speed [50].

9. Acknowledgements

This work was supported by MCTIC/CNPq-FAPESQ/PB (EDITAL N° 010/2021) and by VTEX BRASIL (EMBRAPII PCEE-2304.0229).

References

- [1] H. Qiu, S. Jha, S. S. Banerjee, A. Patke, C. Wang, H. Franke, Z. Kalbarczyk, R. Iyer, Is function-as-a-service a good fit for latency-critical services?, *Proceedings of the Seventh International Workshop on Serverless Computing (WoSC7) 2021* (2021). doi:10.1145/3493651.3493666.
- [2] A. Passwater, 2018 serverless community survey: huge growth in serverless usage, Available at: <https://www.serverless.com/blog/2018-serverless-community-survey-huge-growth-usage/>, accessed: 2022-02-02.
- [3] Amazon, Serverless on aws, Available at: <https://aws.amazon.com/serverless/>, accessed: 2022-02-02.
- [4] Google, Serverless computing, Available at: <https://cloud.google.com/serverless>, accessed: 2022-02-02.
- [5] Microsoft, Azure functions, Available at: <https://azure.microsoft.com/en-us/services/functions/>, accessed: 2022-02-02.
- [6] IBM, Ibm cloud functions, Available at: <https://www.ibm.com/cloud/functions>, accessed: 2022-02-02.
- [7] Huawei, Cloud functions, Available at: <https://developer.huawei.com/consumer/en/agconnect/cloud-function/>, accessed: 2022-02-02.
- [8] E. Jonas, J. Schleier-Smith, V. Sreekanti, C. Tsai, A. Khandelwal, Q. Pu, V. Shankar, J. Carreira, K. Krauth, N. J. Yadwadkar, J. E. Gonzalez, R. A. Popa, I. Stoica, D. A. Patterson, Cloud programming simplified: A berkeley view on serverless computing, *CoRR abs/1902.03383* (2019). arXiv:1902.03383. URL <http://arxiv.org/abs/1902.03383>
- [9] P. Lin, A. Glikson, Mitigating cold starts in serverless platforms: A pool-based approach, *CoRR abs/1903.12221* (2019). arXiv:1903.12221. URL <http://arxiv.org/abs/1903.12221>
- [10] Amazon, Creating low-latency, high-volume apis with provisioned concurrency, Available at: <https://aws.amazon.com/fr/blogs/compute/creating-low-latency-high-volume-apis-with-provisioned-concurrency/>, accessed: 2022-02-02.
- [11] I. E. Akkus, R. Chen, I. Rimac, M. Stein, K. Satzke, A. Beck, P. Aditya, V. Hilt, SAND: towards high-performance serverless computing, in: H. S. Gunawi, B. Reed (Eds.), 2018 USENIX Annual Technical Conference, USENIX ATC 2018, Boston, MA, USA, July 11-13, 2018, USENIX Association, 2018, pp. 923–935.
- [12] H. Fingler, A. Akshintala, C. J. Rossbach, USETL: unikernels for serverless extract transform and load why should you settle for less?, in: *Proceedings of the 10th ACM SIGOPS Asia-Pacific Workshop on Systems, APSys 2019, Hangzhou, China, August 19-20, 2019*, ACM, 2019, pp. 23–30. doi:10.1145/3343737.3343750. URL <https://doi.org/10.1145/3343737.3343750>
- [13] J. Thalheim, P. Bhatotia, P. Fonseca, B. Kasikci, Cntr: Lightweight OS containers, in: H. S. Gunawi, B. Reed (Eds.), 2018 USENIX Annual Technical Conference, USENIX ATC 2018, Boston, MA, USA, July 11-13, 2018, USENIX Association, 2018, pp. 199–212. URL <https://www.usenix.org/conference/atc18/presentation/thalheim>
- [14] E. Oakes, L. Yang, D. Zhou, K. Houck, T. Harter, A. C. Arpaci-Dusseau, R. H. Arpaci-Dusseau, SOCK: rapid task provisioning with serverless-optimized containers, in: H. S. Gunawi, B. Reed (Eds.), 2018 USENIX Annual Technical Conference, USENIX ATC 2018, Boston, MA, USA, July 11-13, 2018, USENIX Association, 2018, pp. 57–70.
- [15] A. Agache, M. Brooker, A. Iordache, A. Liguori, R. Neugebauer, P. Piwonka, D. Popa, Firecracker: Lightweight virtualization for serverless applications, in: R. Bhagwan, G. Porter (Eds.), 17th USENIX Symposium on Networked Systems Design and Implementation, NSDI 2020, Santa Clara, CA, USA, February 25-27, 2020, USENIX Association, 2020, pp. 419–434. URL <https://www.usenix.org/conference/nsdi20/presentation/agache>
- [16] J. Cadden, T. Unger, Y. Awad, H. Dong, O. Krieger, J. Appavoo, SEUSS: skip redundant paths to make serverless fast, in: A. Bilas, K. Magoutis, E. P. Markatos, D. Kostic, M. I. Seltzer (Eds.), EuroSys '20: Fifteenth EuroSys Conference 2020, Heraklion, Greece, April 27-30, 2020, ACM, 2020, pp. 32:1–32:15. doi:10.1145/3342195.3392698. URL <https://doi.org/10.1145/3342195.3392698>
- [17] D. Quaresma, D. Fireman, T. Silva, Controlling garbage collection and request admission to improve performance of faas applications, in: *IEEE 32nd International Symposium on Computer Architecture and High Performance Computing (SBAC-PAD)*, 2020. doi:10.1109/SBAC-PAD49847.2020.00033.
- [18] E. V. Eyk, A. Iosup, J. Grohmann, S. Eismann, A. Bauer, L. Versluis, L. Toader, N. Schmitt, N. Herbst, C. L. Abad, The SPEC-RG reference architecture for faas: From microservices and containers to serverless platforms, *IEEE Internet Comput.* 23 (6) (2019) 7–18. doi:10.1109/MIC.2019.2952061. URL <https://doi.org/10.1109/MIC.2019.2952061>
- [19] A. W. Services, Firecracker (may 2018). URL <https://firecracker-microvm.github.io>
- [20] A. Mohan, H. Sane, K. Doshi, S. Edupuganti, N. Nayak, V. Sukhomlinov, Agile cold starts for scalable serverless, in: C. Delimitrou, D. R. K. Ports (Eds.), 11th USENIX Workshop on Hot Topics in Cloud Computing, HotCloud 2019, Renton, WA, USA, July 8, 2019, USENIX Association, 2019.
- [21] O. JDK, Coordinated restore at checkpoint (crac) (jun 2020). URL <https://openjdk.org/projects/crac/>
- [22] J. S. Plank, M. Beck, G. Kingsley, K. Li, Libckpt: Transparent checkpointing under UNIX, in: *USENIX 1995 Technical Conference on UNIX and Advanced Computing Systems*, New Orleans, Louisiana, USA, January 16-20, 1995, Conference Proceedings, USENIX Association, 1995, pp. 213–224. URL <https://www.usenix.org/conference/usenix-1995-technical-conference/libckpt-transparent-checkpointing-under-unix>
- [23] C. Community, Checkpoint/restore in userspace (criu) (jun 2023). URL <https://criu.org>
- [24] L. Kernel, Linux merged patch for unprivileged checkpoint/restore (aug 2020). URL <https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/commit/?id=74858abb103222f922487fd1a24513bbcd80f9>
- [25] S. K. Mohanty, G. Premsankar, M. D. Francesco, An evaluation of open source serverless computing frameworks, in: 2018 IEEE International Conference on Cloud Computing Technology and Science, CloudCom 2018, Nicosia, Cyprus, December 10-13, 2018, IEEE Computer Society, 2018, pp. 115–120. doi:10.1109/CloudCom2018.2018.00033. URL <https://doi.org/10.1109/CloudCom2018.2018.00033>
- [26] H. Lee, K. Satyam, G. C. Fox, Evaluation of production serverless computing environments, in: 11th IEEE International Conference on Cloud Computing, CLOUD 2018, San Francisco, CA, USA, July 2-7, 2018, IEEE Computer Society, 2018, pp. 442–450. doi:10.1109/CLOUD.2018.00062. URL <https://doi.org/10.1109/CLOUD.2018.00062>
- [27] L. Wang, M. Li, Y. Zhang, T. Ristenpart, M. M. Swift, Peeking behind the curtains of serverless platforms, in: H. S. Gunawi, B. Reed (Eds.), 2018 USENIX Annual Technical Conference, USENIX ATC 2018, Boston, MA, USA, July 11-13, 2018, USENIX Association, 2018, pp. 133–146. URL <https://www.usenix.org/conference/atc18/presentation/wang-liang>
- [28] B. Efron, R. J. Tibshirani, An Introduction to the Bootstrap, no. 57 in *Monographs on Statistics and Applied Probability*, Chapman & Hall/CRC, Boca Raton, Florida, USA, 1993.
- [29] S. S. SHAPIRO, M. B. WILK, An analysis of variance test for normality (complete samples), *Biometrika* 52 (3-4) (1965) 591–611. doi:10.1093/biomet/52.3-4.591. URL <https://doi.org/10.1093/biomet/52.3-4.591>
- [30] S. M. Blackburn, K. S. McKinley, R. Garner, C. Hoffmann, A. M. Khan, R. Bentzur, A. Diwan, D. Feinberg, D. Frampton, S. Z. Guyer, M. Hirzel, A. Hosking, M. Jump, H. Lee, J. E. B. Moss, A. Phansalkar, D. Ste-

- fanovik, T. VanDrunen, D. von Dincklage, B. Wiedermann, Wake up and smell the coffee: Evaluation methodology for the 21st century, *Communications of the ACM* 51 (8) (2008) 83–89. doi:10.1145/1378704.1378723.
URL <http://doi.acm.org/10.1145/1378704.1378723>
- [31] F. Bellard, QEMU, a fast and portable dynamic translator, in: 2005 USENIX Annual Technical Conference (USENIX ATC 05), USENIX Association, Anaheim, CA, 2005.
- [32] E. Casalicchio, V. Perciballi, Measuring docker performance : What a mess!!!, in: ICPE 2017 - Companion of the 2017 ACM/SPEC International Conference on Performance Engineering :, ICPE '17 Companion, ACM, 2017, pp. 11–16. doi:10.1145/3053600.3053605.
URL <http://doi.acm.org/10.1145/3053600.3053605>
- [33] M. Raho, A. Spyridakis, M. Paolino, D. Raho, Kvm, xen and docker: A performance analysis for arm based nfv and cloud computing, in: IEEE 3rd Workshop on Advances in Information, Electronic and Electrical Engineering (AIEEE), 2015, pp. 1–8. doi:10.1109/AIEEE.2015.7367280.
- [34] Docker, Docker project page, Available at: <https://www.docker.com>, accessed: 2022-02-02.
- [35] Y. Li, Y. Lin, Y. Wang, K. Ye, C. Xu, Serverless computing: State-of-the-art, challenges and opportunities, *IEEE Trans. Serv. Comput.* 16 (2) (2023) 1522–1539. doi:10.1109/TSC.2022.3166553.
URL <https://doi.org/10.1109/TSC.2022.3166553>
- [36] E. Oakes, L. Yang, K. Houck, T. Harter, A. C. Arpaci-Dusseau, R. H. Arpaci-Dusseau, Pipsqueak: Lean lambdas with large libraries, in: A. Musae, J. E. Ferreira, T. Higashino (Eds.), 37th IEEE International Conference on Distributed Computing Systems Workshops, ICDCS Workshops 2017, Atlanta, GA, USA, June 5-8, 2017, IEEE Computer Society, 2017, pp. 395–400. doi:10.1109/ICDCSW.2017.32.
URL <https://doi.org/10.1109/ICDCSW.2017.32>
- [37] LXC, Lxc project page, Available at: <https://linuxcontainers.org/lxc/>, accessed: 2022-02-02.
- [38] S. Boucher, A. Kalia, D. G. Andersen, M. Kaminsky, Putting the "micro" back in microservice, in: H. S. Gunawi, B. Reed (Eds.), 2018 USENIX Annual Technical Conference, USENIX ATC 2018, Boston, MA, USA, July 11-13, 2018, USENIX Association, 2018, pp. 645–650.
- [39] Amazon, Starting up faster with aws lambda snapstart, Available at: <https://aws.amazon.com/pt/blogs/compute/starting-up-faster-with-aws-lambda-snapstart/>, accessed: 2022-02-02.
- [40] P. Silva, D. Fireman, T. E. Pereira, Prebaking functions to warm the serverless cold start, in: Proceedings of the 21st International Middleware Conference, Middleware '20, Association for Computing Machinery, New York, NY, USA, 2020, p. 1–13. doi:10.1145/3423211.3425682.
URL <https://doi.org/10.1145/3423211.3425682>
- [41] S. Kukreti, F. Mueller, Clonohadoop: Process cloning to reduce hadoop's long tail, in: 5th IEEE/ACM International Conference on Big Data Computing Applications and Technologies, BDCAT 2018, Zurich, Switzerland, December 17-20, 2018, IEEE Computer Society, 2018, pp. 11–20. doi:10.1109/BDCAT.2018.00011.
URL <https://doi.org/10.1109/BDCAT.2018.00011>
- [42] K. Kawachiya, K. Ogata, D. Silva, T. Onodera, H. Komatsu, T. Nakatani, Cloneable JVM: a new approach to start isolated java applications faster, in: C. Krintz, S. Hand, D. Tarditi (Eds.), Proceedings of the 3rd International Conference on Virtual Execution Environments, VEE 2007, San Diego, California, USA, June 13-15, 2007, ACM, 2007, pp. 1–11. doi:10.1145/1254810.1254812.
URL <https://doi.org/10.1145/1254810.1254812>
- [43] J. Oh, S. Moon, Snapshot-based loading-time acceleration for web applications, in: K. Olukotun, A. Smith, R. Hundt, J. Mars (Eds.), Proceedings of the 13th Annual IEEE/ACM International Symposium on Code Generation and Optimization, CGO 2015, San Francisco, CA, USA, February 07 - 11, 2015, IEEE Computer Society, 2015, pp. 179–189. doi:10.1109/CGO.2015.7054198.
URL <https://doi.org/10.1109/CGO.2015.7054198>
- [44] S. Shillaker, P. Pietzuch, Faasm: Lightweight isolation for efficient stateful serverless computing, in: 2020 USENIX Annual Technical Conference (USENIX ATC 20), USENIX Association, 2020, pp. 419–433.
URL <https://www.usenix.org/conference/atc20/presentation/shillaker>
- [45] D. Ustiugov, P. Petrov, M. Kogias, E. Bugnion, B. Grot, Benchmarking, Analysis, and Optimization of Serverless Function Snapshots, Association for Computing Machinery, New York, NY, USA, 2021, p. 559–572.
URL <https://doi.org/10.1145/3445814.3446714>
- [46] D. K. Kim, H.-G. Roh, Scheduling containers rather than functions for function-as-a-service, in: 2021 IEEE/ACM 21st International Symposium on Cluster, Cloud and Internet Computing (CCGrid), 2021, pp. 465–474. doi:10.1109/CCGrid51090.2021.00056.
- [47] A. Fuerst, P. Sharma, Faas-cache: Keeping serverless computing alive with greedy-dual caching, in: Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, ASPLOS 2021, Association for Computing Machinery, New York, NY, USA, 2021, p. 386–400. doi:10.1145/3445814.3446757.
URL <https://doi.org/10.1145/3445814.3446757>
- [48] S. Agarwal, M. A. Rodriguez, R. Buyya, A reinforcement learning approach to reduce serverless function cold start frequency, in: 2021 IEEE/ACM 21st International Symposium on Cluster, Cloud and Internet Computing (CCGrid), 2021, pp. 797–803. doi:10.1109/CCGrid51090.2021.00097.
- [49] J. Manner, M. EndreB, T. Heckel, G. Wirtz, Cold start influencing factors in function as a service, in: A. Sill, J. Spillner (Eds.), 2018 IEEE/ACM International Conference on Utility and Cloud Computing Companion, UCC Companion 2018, Zurich, Switzerland, December 17-20, 2018, IEEE, 2018, pp. 181–188. doi:10.1109/UCC-Companion.2018.00054.
URL <https://doi.org/10.1109/UCC-Companion.2018.00054>
- [50] R. S. Venkatesh, T. Smejkal, D. S. Milojevic, A. Gavrilovska, Fast in-memory CRIU for docker containers, in: Proceedings of the International Symposium on Memory Systems, MEMSYS 2019, Washington, DC, USA, September 30 - October 03, 2019, ACM, 2019, pp. 53–65. doi:10.1145/3357526.3357542.
URL <https://doi.org/10.1145/3357526.3357542>

Author Biography

Daniel Fireman has been a Federal Institute of Alagoas professor since 2017. He holds a Ph.D. in Computer Science from the Federal University of Campina Grande and spent six years working at Google. He researches resource management, capacity planning, and performance engineering. He has also been a civic hacker, leading the development of dadosjusbr.org platform and advising projects on government transparency.

Author Biography

[Click here to access/download;Author Photo;foto_perfil.jpeg](#) 



Daniel Fireman: Conceptualization, Methodology, Data Curation, Writing - Original Draft. Paulo Silva: Conceptualization, Investigation, Software, Writing - Review & Editing, Visualization. Thiago Emmanuel Pereira: Supervision, Methodology, Writing - Review & Editing, Funding acquisition. Luis Mafra: Investigation, Software, Writing - Review & Editing, Visualization. Dalton Cézane Gomes Valadares: Investigation, Writing - Review & Editing, Visualization

Declaration of Interest Statement

Declaration of interests

☐The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

☒The authors declare the following financial interests/personal relationships which may be considered as potential competing interests:

Thiago Emmanuel Pereira reports financial support was provided by Research Support Foundation of Paraíba State. Thiago Emmanuel Pereira reports financial support was provided by VTEX. Thiago Emmanuel Pereira reports financial support was provided by Ministry of Science Technology and Innovations. Thiago Emmanuel Pereira reports financial support was provided by National Council for Scientific and Technological Development.